

Destructor:

- Destructor is an instance member function of a class.
- The name of the destructor is same as the name of a class but preceded by tilde (~) symbol.

Destructor can never be static.

Destructor has no return type.

Destructor takes no argument (No overloading is possible)

- * It is invoked implicitly when object is going to destroy.

Program: #include <iostream>

using namespace std;

Class complex

{

private:

int a, b;

public:

Complex()

{

cout << "Destructor";

}

};

void fun()

{

Complex obj;

}

void main()

{

fun();

}

*

Object ko destroy hone wala hota
to usse just phale destructor
chalta hai.

In Console ÷

Destructor (It will display)



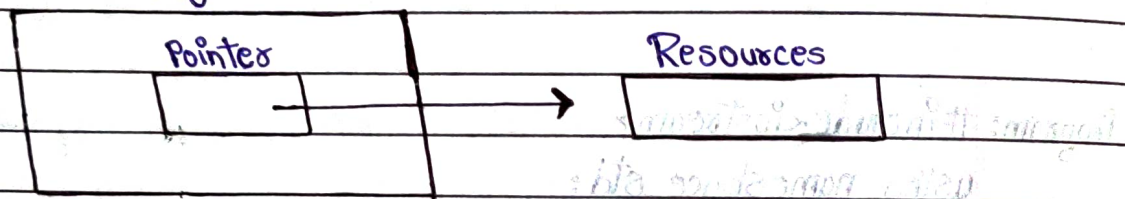
Why destructor?

As we know destructor invoked implicitly just before destroy of object.

So any task that we have to do just before object getting destroy or closed we can perform it by using it like ÷

It should be defined to release resources allocated to an object

Example ÷



Suppose we have an pointer inside object pointing to an address. So if the object get destroy we will not able to access that address again.

So, Destructor can be used to free() that address memory.